

0.1 Motivation

Previous discussion, while giving certain applications of the Sum-Check protocol, still provides quite “artificial” constructions not directly applicable in the real-world use-cases. For instance, the #SAT problem is not very practical as the prover $\mathcal{P}$ cannot even compute the value H in polynomial time. Ideally, we would like to have a protocol that allows the prover to compute the value H in the polynomial time, while the verifier $\mathcal{V}$ should be able to verify the correctness of the claimed value H in the logarithmic time.

Goldwasser, Kalai, and Rothblum (GKR) [?] described a protocol which solves exactly this issue over the arithmetical circuits, which we solved using $\text{QAP} \rightarrow \text{NILP}$ reduction in Groth16. Here we take the Sum-Check approach.

Suppose we are given the *layered* arithmetical circuit $\mathcal{C} : \mathbb{F}^n \rightarrow \mathbb{F}^m$ of size S (number of gates). The *layered* here means that the circuit $\mathcal{C}$ can be decomposed into d layers (note that GKR can be generalized to the unstructured arithmetical circuits as well). The GKR protocol allows to achieve the following performance:

- The communication consists of $\mathcal{O}(d \cdot \text{polylog}(S))$ field elements.
- The verifier runs in $\mathcal{O}(n + d \cdot \text{polylog}(S))$ time.
- The prover runs in $\mathcal{O}(\text{poly}(S))$ time.
- The soundness error is just $\mathcal{O}(d \log(S)/|\mathbb{F}|)$.

0.2 Protocol Description

0.2.1 Circuit Representation

Again, assume we are given the circuit $\mathcal{C} : \mathbb{F}^n \rightarrow \mathbb{F}^m$ of size S , depth d , and fan-in two (i.e., each gate has at most two inputs, but might have multiple outputs). Let us number the layers of the circuit from 0 to d where 0 denotes the output layer while d is the input layer (notice that the layers are numbered in the reverse order compared to the usual circuit notation). Assume the number of gates in the layer i is S_i and without the loss of generality, we assume that S_i is the power of two: $S_i = 2^{v_i}$. Now, we are going to introduce certain functions to further build up the Sum-Check.

Gates Encodings. Suppose $W_i : \{0, 1\}^{v_i} \rightarrow \mathbb{F}$ is structured so that it outputs the value of the i -th layer gate given the gate label. As usual, we denote the multilinear extension of W_i as $\widehat{W}_i : \mathbb{F}^{v_i} \rightarrow \mathbb{F}$.

Wiring Predicates. We introduce the wiring predicates $\text{in}_{1,i}, \text{in}_{2,i} : \{0, 1\}^{v_i} \rightarrow \{0, 1\}^{v_{i+1}}$ which indicate which pairs of wiring are connected to the i -th layer gate from the layer $i + 1$. It takes the label of the gate in the layer i (say, a) and the function $\text{in}_{1,i}(a)$ outputs the label of the first input connected to the gate a in the layer $i + 1$, while $\text{in}_{2,i}(a)$ outputs the label of the second input connected to the gate a in the layer $i + 1$.

Operations Encodings. Define two functions: $\text{add}, \text{mul} : \{0, 1\}^{v_i + 2v_{i+1}} \rightarrow \{0, 1\}$ which take three gate labels (say, (a, b, c)) and return 1 if and only if

$(b, c) = (\text{in}_{1,i}(a), \text{in}_{2,i}(a))$ (that is, the gate a in the layer i is connected to the gates b and c in the layer $i + 1$) and a is an addition gate. Similarly, $\text{mul}(a, b, c)$ returns 1 if and only if $(b, c) = (\text{in}_{1,i}(a), \text{in}_{2,i}(a))$ and a is a multiplication gate. We denote the MLEs of these functions as $\widetilde{\text{add}}$ and $\widetilde{\text{mul}} : \mathbb{F}^{v_i + 2v_{i+1}} \rightarrow \mathbb{F}$, respectively.

Example. Consider the following circuit with $d = 3$ layers:

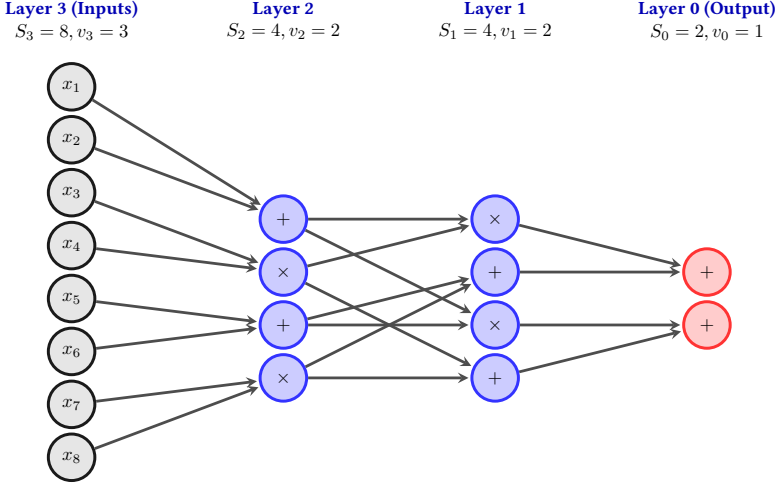


Figure 0.1: Example of a layered arithmetical circuit $C : \mathbb{F}^8 \rightarrow \mathbb{F}^2$ with $d = 3$ layers.

Let us assume that we have inputted the values $\mathbf{x}^{(3)} = (1, 2, 0, 1, 0, 2, 0, 1)$. Then, the value of the layer 2 is $\mathbf{x}^{(2)} = (3, 0, 2, 0)$ and the layer 1 is $\mathbf{x}^{(1)} = (0, 2, 6, 0)$. Finally, the output layer is $\mathbf{x}^{(0)} = (2, 6)$. Let us now see how to encode gates, wiring predicates, and operations encodings on this circuit. Consider $W_1 : \{0, 1\}^2 \rightarrow \mathbb{F}$. We have

$$W_1(0, 0) = 0, \quad W_1(0, 1) = 2, \quad W_1(1, 0) = 6, \quad W_1(1, 1) = 0.$$

Therefore, the multilinear extension of W_1 is $\widetilde{W}_1(X_1, X_2) = 2(1 - X_1)X_2 + 6X_1(1 - X_2)$.

Now, let us define the wiring predicates $\text{in}_{1,2}, \text{in}_{2,2} : \{0, 1\}^2 \rightarrow \{0, 1\}^3$. Consider the 2nd gate ($2 = (1, 0)$) in layer 2. It is connected to the 5th and 6th gates in layer 3 (the input layer). Since $5 = (1, 0, 1)$ and $6 = (1, 1, 0)$ in the binary form, we have

$$\text{in}_{1,2}(1, 0) = (1, 0, 1), \quad \text{in}_{2,2}(1, 0) = (1, 1, 0).$$

Now, consider the operation encodings add_2 and $\text{mul}_2 : \{0, 1\}^8 \rightarrow \{0, 1\}$. The addition gate is non-zero only on the following inputs:

$$((0, 0), (0, 0, 0), (0, 0, 1)), \quad ((1, 0), (1, 0, 0), (1, 0, 1)).$$

Therefore, the multilinear extension of add_2 is

$$\begin{aligned}\widetilde{\text{add}}_2(X_1, X_2, Y_1, Y_2, Y_3, Z_1, Z_2, Z_3) &= (1 - X_1)(1 - X_2)(1 - Y_1)(1 - Y_2) \\ &\quad \cdot (1 - Y_3)(1 - Z_1)(1 - Z_2)Z_3 \\ &\quad + X_1(1 - X_2)Y_1(1 - Y_2)(1 - Y_3)Z_1(1 - Z_2)Z_3\end{aligned}$$

Similarly, the multiplication encoding mul_2 is non-zero only on the following inputs:

$$((0, 1), (0, 1, 0), (0, 1, 1)), \quad ((1, 1), (1, 1, 0), (1, 1, 1)),$$

The multilinear extension of mul_2 is

$$\begin{aligned}\widetilde{\text{mul}}_2(X_1, X_2, Y_1, Y_2, Y_3, Z_1, Z_2, Z_3) &= (1 - X_1)X_2(1 - Y_1)Y_2(1 - Y_3)(1 - Z_1)Z_2Z_3 \\ &\quad + X_1X_2Y_1Y_2(1 - Y_3)Z_1Z_2Z_3.\end{aligned}$$

Remark. Note that the operations encodings add_i and mul_i (and thus MLEs $\widetilde{\text{add}}_i$ and $\widetilde{\text{mul}}_i$) do not depend on the solution witness $\{\mathbf{x}^{(i)}\}_{i \in [d+1]}$, while the gates encodings W_i do depend.

0.2.2 Protocol Specification

The GKR protocol consists of d rounds, one for each layer of the circuit. In the i -th round, the prover $\mathcal{P}$ claims a value for $\widetilde{W}_i(\mathbf{r}_i)$ at the randomly selected point $\mathbf{r}_i \leftarrow \$ \mathbb{F}^{v_i}$.

At the start of the 1st round, this claim is made about the circuit outputs. Namely, if we have $S_0 = 2^{v_0}$ gates in the output layer, let $D : \{0, 1\}^{v_0} \rightarrow \mathbb{F}$ denote the function which maps the gate label to the claimed output value, sent by the prover $\mathcal{P}$. The verifier picks a random point $\mathbf{r}_0 \leftarrow \$ \mathbb{F}^{v_0}$ and evaluates $\widetilde{D}(\mathbf{r}_0)$ in time $\mathcal{O}(S_0)$. By the Schwartz-Zippel lemma, if $\widetilde{D}(\mathbf{r}_0) = \widetilde{W}_0(\mathbf{r}_0)$, then with the soundness error of $v_0/|\mathbb{F}|^{v_0}$ the verifier is assured that these two polynomials are indeed equal. However, for obvious reasons, the verifier cannot compute $\widetilde{W}_0(\mathbf{r}_0)$ without the prover's help. So the core idea of GKR is to reduce the claim about the value of $\widetilde{W}_i(\mathbf{r}_i)$ to a claim about the value of $\widetilde{W}_{i+1}(\mathbf{r}_{i+1})$ for some randomly chosen $\mathbf{r}_{i+1} \leftarrow \$ \mathbb{F}^{v_{i+1}}$. How? Consider the following lemma.

Lemma 0.1. The following statement holds:

$$\begin{aligned}\widetilde{W}_i(\mathbf{z}) &= \sum_{\mathbf{b}, \mathbf{c} \in \{0, 1\}^{v_{i+1}}} \left(\widetilde{\text{add}}_i(\mathbf{z}, \mathbf{a}, \mathbf{b})(\widetilde{W}_{i+1}(\mathbf{b}) + \widetilde{W}_{i+1}(\mathbf{c})) \right. \\ &\quad \left. + \widetilde{\text{mul}}_i(\mathbf{z}, \mathbf{b}, \mathbf{c})\widetilde{W}_{i+1}(\mathbf{b})\widetilde{W}_{i+1}(\mathbf{c}) \right).\end{aligned}$$

Proof. Since both sides are multilinear polynomials, it suffices to check that the equality holds for all boolean assignments $\mathbf{z} \in \{0, 1\}^{v_i}$. Fix some concrete

$\mathbf{z}_0 \in \{0, 1\}^{v_i}$ and without the loss of generality, assume that $\mathbf{z}_0$ is the addition gate in the layer i (the case of multiplication gate is similar). In such case, all the terms $\widetilde{\text{mul}}_i(\mathbf{z}, \mathbf{b}, \mathbf{c})$ will be zero, so we can reduce the sum down to:

$$\widetilde{W}_i(\mathbf{z}_0) = \sum_{\mathbf{b}, \mathbf{c} \in \{0, 1\}^{v_{i+1}}} \widetilde{\text{add}}_i(\mathbf{z}_0, \mathbf{a}, \mathbf{b})(\widetilde{W}_{i+1}(\mathbf{b}) + \widetilde{W}_{i+1}(\mathbf{c}))$$

Now, according to the definition of the $\widetilde{\text{add}}_i$ predicate, the $\widetilde{\text{add}}_i(\mathbf{z}_0, \mathbf{b}, \mathbf{c})$ is non-zero (equals to 1) only if $(\mathbf{b}, \mathbf{c}) = (\text{in}_{1,i}(\mathbf{z}_0), \text{in}_{2,i}(\mathbf{z}_0))$. Since we know that $\mathbf{z}_0$ is an addition gate, we can conclude that two such gates $\mathbf{b}$ and $\mathbf{c}$ exist. Therefore, we can rewrite the sum as:

$$\widetilde{W}_{i+1}(\text{in}_{1,i}(\mathbf{z}_0)) + \widetilde{W}_{i+1}(\text{in}_{2,i}(\mathbf{z}_0)) = \widetilde{W}_i(\mathbf{z}_0).$$

That said, to check the $\mathcal{P}$'s claim about $\widetilde{W}_i(\mathbf{r}_i)$, the verifier can apply the Sum-Check protocol on the function

$$f_i(\mathbf{b}, \mathbf{c}; \mathbf{r}_i) = \widetilde{\text{add}}_i(\mathbf{r}_i, \mathbf{b}, \mathbf{c})(\widetilde{W}_{i+1}(\mathbf{b}) + \widetilde{W}_{i+1}(\mathbf{c})) + \widetilde{\text{mul}}_i(\mathbf{r}_i, \mathbf{b}, \mathbf{c})\widetilde{W}_{i+1}(\mathbf{b})\widetilde{W}_{i+1}(\mathbf{c}).$$

However, note that *the verifier $\mathcal{V}$ does not know the polynomial $\widetilde{W}_{i+1}$* . However, note that he does not need to know it until the last round, during which $\mathcal{V}$ has to make the oracle request $\mathcal{O}^{f_i}$ to compute the value of f_i at the randomly selected point $(\mathbf{b}^*, \mathbf{c}^*) \leftarrow_{\$} \mathbb{F}^{2v_{i+1}}$. Evaluating $f_i(\mathbf{b}^*, \mathbf{c}^*; \mathbf{r}_i)$ requires evaluating $\widetilde{\text{add}}_i(\mathbf{r}_i, \mathbf{b}^*, \mathbf{c}^*)$, $\widetilde{\text{mul}}_i(\mathbf{r}_i, \mathbf{b}^*, \mathbf{c}^*)$, and $\widetilde{W}_{i+1}(\mathbf{b}^*)$ with $\widetilde{W}_{i+1}(\mathbf{c}^*)$. Note that evaluating the first two terms can be done by the verifier $\mathcal{V}$ in $\mathcal{O}(\text{poly}(v_i, v_{i+1}))$. However, evaluating the last two terms requires the prover $\mathcal{P}$ to assist the verifier. So we are going to do the following: the prover $\mathcal{P}$ sends the values $z_b = \widetilde{W}_{i+1}(\mathbf{b}^*)$ and $z_c = \widetilde{W}_{i+1}(\mathbf{c}^*)$ to the verifier $\mathcal{V}$. Now the verifier has to check both these values are correct. However, here is the issue: during the sum-check protocol, the verifier can only use a single random point $\mathbf{r}_{i+1} \in \mathbb{F}^{v_{i+1}}$. So how do we check both conditions using one random point? Here is the trick.

Proposition 0.2. Let $\ell : \mathbb{F} \rightarrow \mathbb{F}^{v_{i+1}}$ be the line such that $\ell(0) = \mathbf{b}^*$ and $\ell(1) = \mathbf{c}^*$. Then, the prover $\mathcal{P}$ sends the univariate polynomial $q(X)$ claimed to be equal to $\widetilde{W}_{i+1} \circ \ell$ – the restriction of $\widetilde{W}_{i+1}$ to the line ℓ . $\mathcal{V}$ checks whether indeed $\ell(0) = z_b$ and $\ell(1) = z_c$, then chooses a random point $\mathbf{r}^* \leftarrow_{\$} \mathbb{F}^{v_{i+1}}$ and checks whether $\widetilde{W}_{i+1}(\ell(\mathbf{r}^*)) = q(\mathbf{r}^*)$.

This way, the interaction between the prover and the verifier at this round ends with the new claim about the value of $\widetilde{W}_{i+1}(\mathbf{r}_{i+1})$ with $\mathbf{r}_{i+1} := \ell(\mathbf{r}^*)$.

Finally, we need to define the last round of the protocol. Here, the $\mathcal{V}$ simply evaluates $\widetilde{W}_d(\mathbf{r}_d)$ on his own, costing $\mathcal{O}(n)$ time.

Proposition 0.3. We finally summarize the GKR protocol:

- At the start of the first round, $\mathcal{P}$ sends a function $D : \{0, 1\}^{v_0} \rightarrow \mathbb{F}$ claimed to equal W_0 (that is, he essentially sends the values of the output layer gates).
- $\mathcal{V}$ picks a random point $\mathbf{r}_0 \leftarrow \$ \mathbb{F}^{v_0}$ and finds $m_0 \leftarrow \widetilde{D}(\mathbf{r}_0)$.
- For each $i \in [d]$ we do the following:
 - Define the $2v_i$ -variate polynomial:

$$f_i(\mathbf{b}, \mathbf{c}; \mathbf{r}_i) = \widetilde{\text{add}}_i(\mathbf{r}_i, \mathbf{b}, \mathbf{c})(\widetilde{W}_{i+1}(\mathbf{b}) + \widetilde{W}_{i+1}(\mathbf{c})) \\ + \widetilde{\text{mul}}_i(\mathbf{r}_i, \mathbf{b}, \mathbf{c})\widetilde{W}_{i+1}(\mathbf{b})\widetilde{W}_{i+1}(\mathbf{c}).$$

- $\mathcal{P}$ sends the value m_i claimed to equal $\sum_{\mathbf{b}, \mathbf{c} \in \{0, 1\}^{v_{i+1}}} f_i(\mathbf{b}, \mathbf{c}; \mathbf{r}_i)$.
- $\mathcal{V}$ applies the Sum-Check protocol on the function $f_i(\cdot; \mathbf{r}_i)$ up until the last round, where the $\mathcal{V}$ has to compute the value $f_i(\mathbf{b}^*, \mathbf{c}^*; \mathbf{r}_i)$ at random $(\mathbf{b}^*, \mathbf{c}^*) \in \mathbb{F}^{2v_{i+1}}$.
- Both prover and the verifier computes the line $\ell : \mathbb{F} \rightarrow \mathbb{F}^{v_{i+1}}$ such that $\ell(0) = \mathbf{b}^*$ and $\ell(1) = \mathbf{c}^*$. $\mathcal{P}$ sends the univariate polynomial $q(X)$ claimed to equal $\widetilde{W}_{i+1} \circ \ell$.
- $\mathcal{V}$ computes the required value $f_i(\mathbf{b}^*, \mathbf{c}^*; \mathbf{r}_i)$ using $q(0)$ and $q(1)$ instead of $\widetilde{W}_{i+1}(\mathbf{b}^*)$ and $\widetilde{W}_{i+1}(\mathbf{c}^*)$.
- $\mathcal{V}$ chooses a random point $\mathbf{r}^* \leftarrow \$ \mathbb{F}^{v_{i+1}}$ and sets $\mathbf{r}_{i+1} \leftarrow \ell(\mathbf{r}^*)$ and $m_{i+1} \leftarrow q(\mathbf{r}_{i+1})$.
- The check reduces to verifying that $\widetilde{W}_{i+1}(\mathbf{r}_{i+1}) = m_{i+1}$, so proceed to the next step.
- At the last round, $\mathcal{V}$ directly checks whether $m_d = \widetilde{W}_d(\mathbf{r}_d)$.